Week 4: Some Background Information

154

Neil Cherry, BS-CS
Joseph E. Wilkes, PhD-EE

September 2026

Questions from Last Week

Background

- The last three weeks was designed to provide sufficient information to start planning and executing on the class project
- Other useful information that will be discussed this week
  - AI Systems Comparison *
  - Context (Workspace) Management *
  - Security of AI Generated Code
  - Local AI Systems
- Lab work

* These two sections were created in August 2026 by Claude
Before making decisions based on this material, please perform an updated search for Information

No vendor is the right answer for every job. This module compares six options on the axes that actually change your decision: where your code goes, what it costs, and how much control you keep.

AI CODING ASSISTANTS

157

A brief, comparison

Copilot · Cursor · Codex · Claude Code · Gemini CLI · Local models

As of August 2026

Note: This section was written by Claude after
We asked Claude to review the outline for missing information
Please review updated information before making any decisions on which AI system to use

September 2026

Why these six were chosen

SELECTION

Built to cover distinct interaction models, not to rank products — if two entries answered every question alike, one would be redundant.

How you work with it	Represented by
Inline completion inside your existing IDE	GitHub Copilot
An AI-native editor you migrate to	Cursor
A delegated cloud agent you review async	OpenAI Codex
One agent across terminal, IDE, and CI	Claude Code
An open-source, scriptable terminal agent	Gemini CLI
Inference you host yourself	Local open-weight models

WHAT IS DELIBERATELY NOT HERE

- Windsurf, Amazon Q Developer, JetBrains AI, Tabnine, Zed, Aider and Continue.dev are left out for time, not merit.
- No benchmark ranking — leaderboards move monthly and do not predict fit for your codebase.
- No sponsorship or affiliation with any vendor listed.
- Every entry is in real production use today.

Five model vendors plus open weights — Microsoft/OpenAI routing, model-agnostic, OpenAI, Anthropic, Google, and open-weight communities are all represented, so no single lab's roadmap sets the agenda.

Note: This is a representative sample not an in-depth analysis of all AI Systems!

Four questions that decide the tool

FRAMING

1 **Where does your code go?**
Cloud inference means source leaves your network. Some vendors contractually promise no retention or training. Only local inference makes the question moot.

2 **What is the cost model?**
In 2026 nearly everyone moved from flat seats to token or credit metering. A $20 plan is a floor, not a ceiling. Model your heavy users, not your average one.

3 **Where does it live?**
Autocomplete in your existing IDE, a forked editor you must migrate to, or a terminal agent you script. This drives adoption more than model quality does.

4 **Who owns the failure?**
Hosted tools give you support, IP indemnification, and someone to call. Self-hosted gives you control and hands you the pager.

Model quality is the fifth question.
It is deliberately not on this list — the frontier models are close enough that fit matters more.

GitHub Copilot

TOOL 1 OF 6

The default. Broadest IDE reach, deepest GitHub integration, most conservative enterprise story.

WHERE IT WINS

- Runs everywhere you already work: VS Code, Visual Studio, JetBrains, Neovim, Xcode — no editor migration.
- Tightest GitHub loop: PR review, issue-to-PR agents, Actions integration.
- Enterprise plans include IP indemnification, admin policy controls, and a contractual no-training commitment on your code.
- Inline completions are free of credit metering on every paid plan.

GitHub Copilot — trade-offs and fit

WHERE IT COSTS YOU

- Moved to usage-based AI Credits on 1 June 2026 — base prices held, but heavy agent users now hit unbudgeted overage.
- Agent mode is competent but generally trails purpose-built agents on multi-file refactors.
- You are inside Microsoft's plumbing; model choice and routing are theirs, not yours.
- Cloud-only. No self-hosted inference option.

PRICING (AUG 2026)

Free · Pro $10 · Pro+ $39 · Max $100 · Business $19/user · Enterprise $39/user

DATA HANDLING

No training on Business/Enterprise code; consumer tiers differ — read the plan

BEST FIT

Mixed-language teams already on GitHub who need one tool procurement will approve

https://github.com/features/copilot

Cursor

TOOL 2 OF 6

A VS Code fork built around the AI. Best-in-class inline prediction; you adopt the whole editor to get it.

WHERE IT WINS

- Tab completion and multi-line prediction are widely considered the strongest available.
- Whole-codebase indexing gives the agent real project context, not just the open file.
- Privacy Mode: code is not stored by model providers or used for training. Business plans enforce it org-wide.
- Model-agnostic — pick the frontier model per task rather than accepting one vendor's.

Note: Cursor was built by a team independent from Microsoft

Cursor — trade-offs and fit

WHERE IT COSTS YOU

- It is an editor swap, not a plugin. Extensions, settings, and muscle memory all migrate — that is the real adoption cost.
- The fork tracks upstream VS Code with a lag; some extensions break.
- Credit-based pricing scales fast. Ultra exists at $200/month because people needed it.
- Codebase indexing uploads embeddings by default — verify Privacy Mode before touching proprietary source.

PRICING (AUG 2026)

Hobby free · Pro $20 · Pro+ $60 · Ultra $200 · Teams $40/user · Enterprise custom

DATA HANDLING

Privacy Mode = no storage, no training; org-enforced on Business tiers

BEST FIT

Individuals and small teams who will trade editor lock-in for the best day-to-day ergonomics

https://cursor.com

OpenAI Codex

TOOL 3 OF 6

Delegation, not autocomplete. You hand it a task and a sandbox, then review the diff.

WHERE IT WINS

- Strong at long-horizon work: give it an issue, get back a branch with a coherent diff.
- One agent across surfaces — CLI, VS Code extension, web, iOS, and Amazon Bedrock as of June 2026.
- Automated PR review and GitHub/Slack integration for team workflows.
- Two billing doors: bundled with ChatGPT plans, or bring an API key and pay raw token rates with no plan ceiling.

OpenAI Codex — trade-offs and fit

WHERE IT COSTS YOU

- Switched from message-based to token-based credits in 2026 — usage is harder to predict and output tokens dominate the bill.
- OpenAI models only. No model shopping.
- Cloud task execution means your repository contents leave your environment.
- Weakest of the five at the tight inline-completion loop — it is not trying to be that tool.

PRICING (AUG 2026)

Bundled with ChatGPT Free/Go/Plus/Pro/Business/Enterprise; Pro 5x $100; or API-key token rates

DATA HANDLING

Business/Enterprise API data is not trained on by default; verify for consumer ChatGPT tiers

BEST FIT

Batch and background work — migrations, test backfill, mechanical refactors you can review async

https://developers.openai.com/codex/

Claude and Claude Code

TOOL 4 OF 6

Anthropic's agentic coder — one engine across terminal, IDE, desktop, web, and CI.

WHERE IT WINS

- One engine everywhere: terminal, VS Code, JetBrains, desktop, and web share one CLAUDE.md, settings, and MCP servers.
- Built for multi-file agentic work — planning, parallel subagents, hooks, and shareable team skills.
- Composable: pipe into it, run it headless in CI, wire it to GitHub Actions or GitLab.
- Deployable through Amazon Bedrock or Google Vertex AI, so inference runs in your own cloud account.

Claude and Claude Code — trade-offs and fit

WHERE IT COSTS YOU

- Anthropic models only — the same lock-in critique that applies to Codex and Gemini CLI.
- No inline tab completion. Many teams run it alongside Copilot rather than instead of it.
- Agentic work burns tokens. Subscription tiers carry session and weekly caps; API rates add up on long tasks.
- Broad file and shell access puts the review burden on you. Easy to over-trust on a large diff.

PRICING (AUG 2026)

Free · Pro $20 · Max 5x $100 · Max 20x $200 · Team Premium seat ~$100 (5 min) · Enterprise custom · or API tokens

DATA HANDLING

No training on business or API data by default; Bedrock/Vertex keeps inference in your own cloud

BEST FIT

Multi-file refactors, test backfill, and CI automation with one agent across terminal, IDE, and pipeline

https://claude.com/product/claude-code · https://code.claude.com/docs

Gemini CLI

TOOL 5 OF 6

Apache 2.0 terminal agent with a 1M-token window. The most scriptable of the hosted options.

WHERE IT WINS

- The client is genuinely open source (Apache 2.0) — you can read it, fork it, and audit what it sends.
- 1M-token context handles whole-repository questions that force other tools to chunk.
- Terminal-first and headless-capable: composes with scripts, CI, and GitHub Actions rather than fighting them.
- Three auth paths — Google account, Gemini API key, or Vertex AI for enterprise controls and residency.

Gemini CLI — trade-offs and fit

WHERE IT COSTS YOU

- Free-tier terms have shifted repeatedly. The published 60 req/min and 1,000 req/day figures were reported as curtailed in mid-2026 — verify current quota.
- Open-source client, closed-weight model. The audit stops at the network boundary.
- Gemini models only, and no inline completion — it is an agent, not an in-editor assistant.

PRICING (AUG 2026)

Free tier terms in flux — verify current quota; API key usage-based; Vertex AI for enterprise

DATA HANDLING

Depends entirely on auth path — Vertex AI gives the strongest controls; free tiers the weakest

BEST FIT

Automation, CI pipelines, and large-repo analysis where scriptability beats IDE polish

https://github.com/google-gemini/gemini-cli

Local models

TOOL 6 OF 6

Air-gapped and proprietary work. The only option where the privacy question is closed rather than promised — you pay for it in capability and operations.

WHERE IT WINS

- No code leaves the network. Not a policy, not a contract clause — a network fact. The only entry that satisfies a true air gap.
- No per-token billing and no rate limits after the hardware is bought.
- Works with no internet at all: classified networks, field deployments, bad connectivity.
- Open-weight coding models are now usable on one GPU — a 30B-class MoE runs in roughly 19–24 GB of VRAM.

Local models — trade-offs and fit

WHERE IT COSTS YOU

- A real capability gap on hard reasoning and large refactors. Local models have closed distance, not caught up.
- You own inference ops: serving, upgrades, evals, GPU capacity, and the pager.
- Up-front hardware cost, and it is idle when nobody is coding.
- "Open weights" is not "open source" — several popular licenses restrict commercial use. Read the license.

RUNTIMES

Ollama (easiest start) · llama.cpp (widest hardware) · vLLM (multi-GPU serving)

MODEL FAMILIES

Qwen3-Coder, GLM, Devstral, Gemma and similar open-weight coding models

BEST FIT

Defense, health, finance, and any codebase that legally or contractually cannot be transmitted

ollama.com · github.com/ggml-org/llama.cpp · docs.vllm.ai · huggingface.co/models

Side by side

SUMMARY

	Where code goes	Cost model	Strongest at	Main trade-off
GitHub Copilot	Microsoft cloud; no training on business tiers	Seat + usage credits (since Jun 2026)	Fitting existing IDEs and GitHub	Least control over model routing
Cursor	Vendor cloud; Privacy Mode stops storage	Seat + credits, $0–$200/mo	Inline prediction and editing speed	Editor migration; indexing needs sign-off
OpenAI Codex	OpenAI cloud; sandboxed execution	ChatGPT plans, or API tokens	Delegated, long-running async tasks	Opaque token costs; one model vendor
Claude Code	Anthropic cloud, or your own Bedrock/Vertex	Claude plans $20–$200, or API tokens	Agentic multi-file work across surfaces	No autocomplete; costly per team seat
Gemini CLI	Google cloud; depends on auth path	Free tier (in flux), API key, or Vertex	Scripting, CI, whole-repo analysis	Shifting quota terms; no autocomplete
Local models	Nowhere — stays on your hardware	Hardware capex; no per-token cost	Air-gapped and restricted codebases	Capability gap; you run the infra

Rule of thumb: start from your hardest constraint, not the demo. Data residency → last row. Procurement → first. Developer experience → second.

Where to go next

REFERENCE

IN-EDITOR ASSISTANTS

GitHub Copilot
github.com/features/copilot
Copilot docs
docs.github.com/copilot
Cursor
cursor.com
Cursor docs
docs.cursor.com

AGENTS AND CLIs

OpenAI Codex
developers.openai.com/codex/
Codex CLI source
github.com/openai/codex
Claude Code
claude.com/product/claude-code
Claude Code docs
code.claude.com/docs
Claude apps and plans
claude.com/pricing
Gemini CLI source
github.com/google-gemini/gemini-cli
Gemini CLI docs
geminicli.com/docs/

LOCAL AND SELF-HOSTED

Ollama
ollama.com
llama.cpp
github.com/ggml-org/llama.cpp
vLLM
docs.vllm.ai
Hugging Face hub
huggingface.co/models
Qwen coding models
github.com/QwenLM/Qwen3-Coder
Continue.dev (IDE plugin)
continue.dev
Tabby (completion server)
tabby.tabbyml.com
Aider (any backend)
aider.chat

Prices, quotas, and data-handling terms in this deck were verified in August 2026 and change often — treat every figure here as a starting point for your own check, not a citation.

Token cost is an accounting problem. Context is an engineering problem. This module is about the second one: what occupies the window, why quality falls as it fills, and how to make work survive the session that produced it.

CDL Presents

CONTEXT ENGINEERING

174

Managing the window as a discipline, not an afterthought

Memory files · /compact versus /clear · why long sessions degrade · checkpointing state to disk

August 2026

Note: This section was written by Claude after
We asked Claude to review the outline for missing information
Please review updated information before making any
decisions on how to manage your context within Claude

August 2026

What is Context?

- Context refers to the surrounding information, data, and memory that an AI model uses to interpret a prompt and generate a relevant, accurate response.
- Key Types of Context in AI
  - Conversational History: Past messages and turns in a chat that allow the AI to follow the flow of a conversation without repeating information.
  - System Instructions: Rules, developer guidelines, or role definitions that tell the AI how to behave or format its output.
  - External Data (RAG): Documents, databases, or web searches retrieved and fed to the AI to ground its answers in real facts.
  - User and Situational Data: User preferences, account history, location, or time of day that make an answer personal and specific

Why am We concerned about the Context

- The more information that we type into a session the more data Claude must review as it makes decisions
- If we ask it to do things that are later wrong, it may look at the session and do the wrong thing again
- The bigger the data in the session, the more tokens Claude will use in making decisions
- If the Context get full, Claude will compress/summarize it and may mistakes by keep bad data and removing good data

Why long sessions degrade

Anthropic states it plainly: "Claude's context window fills up fast, and performance degrades as it fills."

1 Context fills

Every message, every file read, and every command output stays in the window. One debugging session can consume tens of thousands of tokens.

2 Signal thins out

Your instructions from turn one now compete with hundreds of lines of failed attempts, stale file contents, and abandoned approaches.

3 Quality falls

Claude starts "forgetting" earlier instructions and making more mistakes — the same model, on a worse prompt.

The counter-intuitive part

A long session is not a better-informed one. Past a point it is a noisier one, and the model cannot tell your live instructions from your abandoned ones.

Corrected the same mistake twice? The window now holds two wrong approaches. A clean start beats a third correction — which is why the fixes ahead are habits, not settings.

What is actually in your context window

You start every session with the window already partly full. Run /context to see the live breakdown for your own session.

Loaded before you type anything

- The system prompt and any output style.
- CLAUDE.md files from the whole directory chain, loaded in full.
- Auto memory: the first 200 lines or 25 KB of MEMORY.md.
- Names and descriptions of every connected MCP tool.
- A one-line description of every available skill.

Grows while you work

- Every prompt you send and every reply you get.
- The full contents of every file Claude reads.
- The complete stdout of every command it runs.
- Path-scoped rules, loaded when a matching file is read.
- Bodies of any skill actually invoked.

Three commands worth memorising

`/context`
Live breakdown by category, plus which memory files actually loaded

`/memory`
Browse and edit every CLAUDE.md and auto-memory file

`/btw`
Ask a side question in an overlay that never enters conversation history

CLAUDE.md and project memory

Every session starts with a fresh window. Two mechanisms carry knowledge across the gap — one you write, one Claude writes.

Where CLAUDE.md files live, in load order

Scope	Location	Purpose
Managed policy	`IT-deployed path`	Org standards. Cannot be excluded.
User	`~/.claude/CLAUDE.md`	Your preferences, every project
Project	`./CLAUDE.md`	Team-shared, committed to git
Local	`./CLAUDE.local.md`	Personal, gitignored

All discovered files are concatenated, not overridden — broadest scope first, so the file closest to where you launched is read last.

Run /init to generate a starter file from your codebase. Run /context to confirm it actually loaded.

Auto memory: the one Claude writes

- Claude saves its own learnings — build commands, debugging insights, preferences it discovers from your corrections.
- Stored per repository at ~/.claude/projects/<project>/memory/, machine-local and not shared across machines.
- MEMORY.md is an index: the first 200 lines or 25 KB load every session. Topic files load only on demand.
- It is plain markdown. Audit it, edit it, delete from it.

Context, not configuration. CLAUDE.md is delivered as a message Claude reads and tries to follow — there is no guaranteed compliance. If something must happen every time, write a hook instead.

Writing a CLAUDE.md that actually gets followed

The test for every line: would removing it cause a mistake? If not, cut it. Bloat is not neutral — it dilutes the rules that matter.

Include

- Build and test commands Claude cannot guess.
- Style rules that differ from language defaults.
- Repository etiquette: branch naming, PR conventions.
- Architectural decisions specific to this project.
- Environment quirks and non-obvious gotchas.

Exclude

- Anything Claude can learn by reading the code.
- Standard conventions the model already knows.
- Detailed API docs — link to them instead.
- File-by-file descriptions of the repository.
- Self-evident advice like "write clean code".

< 200 lines per file is the target. Past that, adherence drops as fast as context fills.

2 wrong answers on the same point means the file is too long or too vague — prune, do not append.

1 source of truth. Contradictory rules across files mean Claude picks one arbitrarily.

/compact versus /clear

One question decides it: do you still need what happened, or are you moving on?

/compact

Same task, too much history

- Replaces the conversation with a structured summary. You keep the thread, you lose the detail.
- Steer it: /compact focus on the auth bug fix keeps what you choose instead of what the automatic pass guesses.
- Runs automatically as you near the limit. /autocompact 500k makes it fire earlier, on your terms.
- Use when the work continues and you want the conclusions without the transcript.

/clear

New task, nothing worth carrying

- Full reset of the conversation window. Memory files reload; the transcript is gone.
- The default move between unrelated tasks. Old conversation crowds out the files you need next.
- Also the right move after two failed corrections: clear, then rewrite the prompt with what you learned.
- A clean session with a better prompt almost always beats a long one full of accumulated corrections.

Third option: don’t put it in the window at all

Delegate the expensive reads. A subagent researches in its own context window and returns only a summary, so a hundred file reads never touch your session at all.

What survives compaction — and what does not

Compaction is lossy and silent. Knowing which mechanism carries an instruction tells you whether it comes back.

Mechanism	After compaction	What that means for you
System prompt and output style	Unchanged	Never part of message history
Project-root CLAUDE.md, unscoped rules	Re-injected from disk	The reason project-level rules are the durable place
Auto memory	Re-injected from disk	Reloaded the same way as CLAUDE.md
Rules with paths: frontmatter	Lost until re-triggered	Returns only when a matching file is read again
Nested CLAUDE.md in subdirectories	Lost until re-triggered	Returns when a file in that subdirectory is read
Invoked skill bodies	Re-injected, but capped	5,000 tokens per skill, 25,000 total; oldest dropped first
Anything said only in chat	Gone	If it matters past this session, it belongs in a file

The rule this produces: an instruction you typed into chat is the least durable thing in your session. If you find yourself repeating it, it has earned a place in CLAUDE.md.

Checkpointing state so work survives the session

Assume every session ends unexpectedly. Anything that only exists in the transcript is already lost.

Write state to disk

- Plan before you build. Have Claude interview you, then write the result to SPEC.md — and start a fresh session to execute it.
- A good spec is self-contained: it names the files and interfaces, states what is out of scope, and ends with a verification step that proves the feature works.
- Keep a running PLAN.md or NOTES.md for long work. It is cheap to re-read and it survives anything.
- Commit early and often. Git is the checkpoint that no tool can lose.

Recover a session

- claude --continue resumes the most recent session; claude --resume picks from a list.
- Name sessions with /rename and treat them like branches — one workstream, one persistent context.
- Esc twice, or /rewind, restores earlier conversation state, code state, or both.
- Rewind also offers Summarize from here and Summarize up to here, so you can compact only part of a session.

Checkpoints are not backups

Rewind only tracks changes made through Claude's file-editing tools. Anything done by a Bash command or an external process is invisible to it. It is a convenience for undoing an approach, not a replacement for git — teach it that way or someone will learn the hard version.

Five failure patterns and their fixes

Every one of these is a context problem wearing a different costume.

Pattern	What it looks like	Fix
The kitchen-sink session	One task, then something unrelated, then back again. The window is now mostly irrelevant.	/clear between unrelated tasks.
Correcting over and over	Two failed corrections leave the context polluted with two wrong approaches.	/clear, then rewrite the prompt with what you learned.
The over-specified CLAUDE.md	The file grew until the rules that matter got lost among the ones that do not.	Prune ruthlessly. Convert must-happen rules to hooks.
The infinite exploration	An unscoped "investigate this" reads hundreds of files straight into your window.	Scope it, or hand it to a subagent.
The trust-then-verify gap	A plausible implementation that nobody checked, produced late in a degraded session.	Give Claude a check it can run, and read the evidence.

The habit loop, in one line

Start clean → put durable facts in CLAUDE.md, not chat → delegate big reads to a subagent → /compact when the task continues, /clear when it does not → write the plan to disk before the window fills.

Risks of Using AI

185



September 2026



Two Wolves Parable



An old Cherokee Indian chief was teaching his grandson about life.

186

"A fight is going on inside me," he told the young boy, "a fight between two wolves. One is evil, full of anger, sorrow, regret, greed, self-pity and false pride. The other is good, full of joy, peace, love, humility, kindness and faith."

"This same fight is going on inside of you, grandson … and inside of every other person on the face of this Earth."

The grandson ponders this for a moment and then asks, "Grandfather, which wolf will win?"

The old man smiled and simply said, "The one you feed."

Question: Is AI Good or is it Bad?

September 2026

CDL

INFOAGE SCIENCE AND HISTORY MUSEUMS

Risks of AI

- Often when we use AI systems, we don't think about what they do with our data
- Most of us do not share personal or company data with others
- But in our leap to use AI, we forget that everything that we do or enter in the AI system is often used to train the AI system to work better
- The following slides talk about those risks and how we can mitigate them
- A later set of slides will show how CDL has been building a local AI system and how it is used to protect data

Unseen Risks

- We are so used to using Personal Computers for our daily work that we often don't think about what happens to our data
- Example Input: "Skynet, Attached is a profit and loss statement for each division of the Acme Corporation for the last 5 years"
- Please create slides plotting quarterly data for the last 5 years:
  - Profit or loss for the entire Acme Corporation
  - Profit or loss for each division of the Acme Corporation
  - A summary slide showing profit and loss for the top 3 and bottom 3 performing divisions
- Have we just lost all of our corporate data that we don't want released?
- Has the AI System just Trained on our Data and no everyone can access it?

Major Risk of Use of Cloud AI services

- When data is stored in the cloud or migrates to the cloud
- Users lose control of their personal information
- Bankruptcy Courts have said the information in the cloud providers data base has value and can be sold to fund bankruptcy settlements

The Root Problem: Loss of Corporate Data Control

- The problem is not the Artificial Intelligence itself,
  - but from losing sovereignty over corporate information once it enters third-party ecosystems
- Standard networks, firewalls, and data storage solutions cannot prevent users from feeding intellectual property into external LLMs
- We can do training but everyone knows people still make mistakes
- This can can cause regulatory issues
- Yes, We are sure everyone is aware of it but how many times do we make a mistake

Duckduckgo

- Duckduckgo has a nice ad running about someone asking about:
  - A face rash
  - Looking for a new job
- Even if done from a phone or from home, how much data about us leaks without us realizing it
- duck.ai claims to be anonymous and not used for training

How can we solve/mitigate the problem

- Build spread sheets or other data sources with default (non-real) data and update the AI results with the real data
- Example: Years ago one of Joe's jobs forced all employees to choose between a cash balance pension or a monthly pension
  - Chose now! And the future you will have to accept your choice! Right or Wrong!
  - The data the company chose to present cherry picked sample employees to show how much better the cash balance was over the monthly pension
  - Joe built a spread sheet testing a bunch of assumptions with his data (age, salary, years working, etc.)
  - Since he was close to retirement, for him, the best solution was the monthly pension
  - Most of his co-workers did not under stand how to analyze the data
  - So, he rebuilt the spread sheet with example data and gave it to many people so they could populate it with their data and thus could decide for them selves

CDL

INFO AGE SCIENCE AND HISTORY MUSEUMS

Best Practices for AI Data Protection

- Opt out of model training:
  - Turn off history and training settings in your AI account preferences so your prompts are not saved to train future models.
- Sanitize and anonymize data:
  - Remove real names, account numbers, project codes, and specific identifiers before submitting text, replacing them with generic placeholders like "Client A".
- Avoid free consumer tiers:
  - Never paste sensitive source code, financials, or legal text into free public AI tools, as inputs may leak or be stored improperly.
- Use enterprise subscriptions:
  - Choose paid business or enterprise-tier tools that explicitly promise data privacy, compliance certifications (like SOC 2 or ISO 27001), and data encryption.
- Deploy private or local models:
  - For extreme data sovereignty, run open-source models locally on your own secure hardware or via dedicated private cloud environments.
- Establish clear team guidelines:
  - Document exact rules for employees on which specific AI tools are approved and what categories of data are prohibited from being shared.

Claude Desktop (Mac mini) Answers the Question: How can I protect my data when using Claude - 1

- Here's what actually moves the needle, in rough order of impact:
- **1. Set the training toggle.** Settings → Privacy in the Claude app.
  - If you don't allow your chats to be used for model improvement, retention stays at 30 days. If you do allow it, new or resumed chats and coding sessions are retained up to five years. You can change it anytime, though data already in an active training run isn't pulled back out retroactively.
- **2. Know the two exceptions to that toggle.** Clicking thumbs up/down stores that entire conversation for five years regardless of your setting.
  - And conversations flagged for policy violations are kept for two years (classifier scores for seven). Neither is affected by opting out.
- **3. Delete what you don't need.**
  - Deleting a chat removes it from history immediately and from backend storage within 30 days. There's no way to un-send something, so deletion is cleanup, not a fix.

Claude Desktop (Mac mini) Answers the Question: How can I protect my data when using Claude - 2

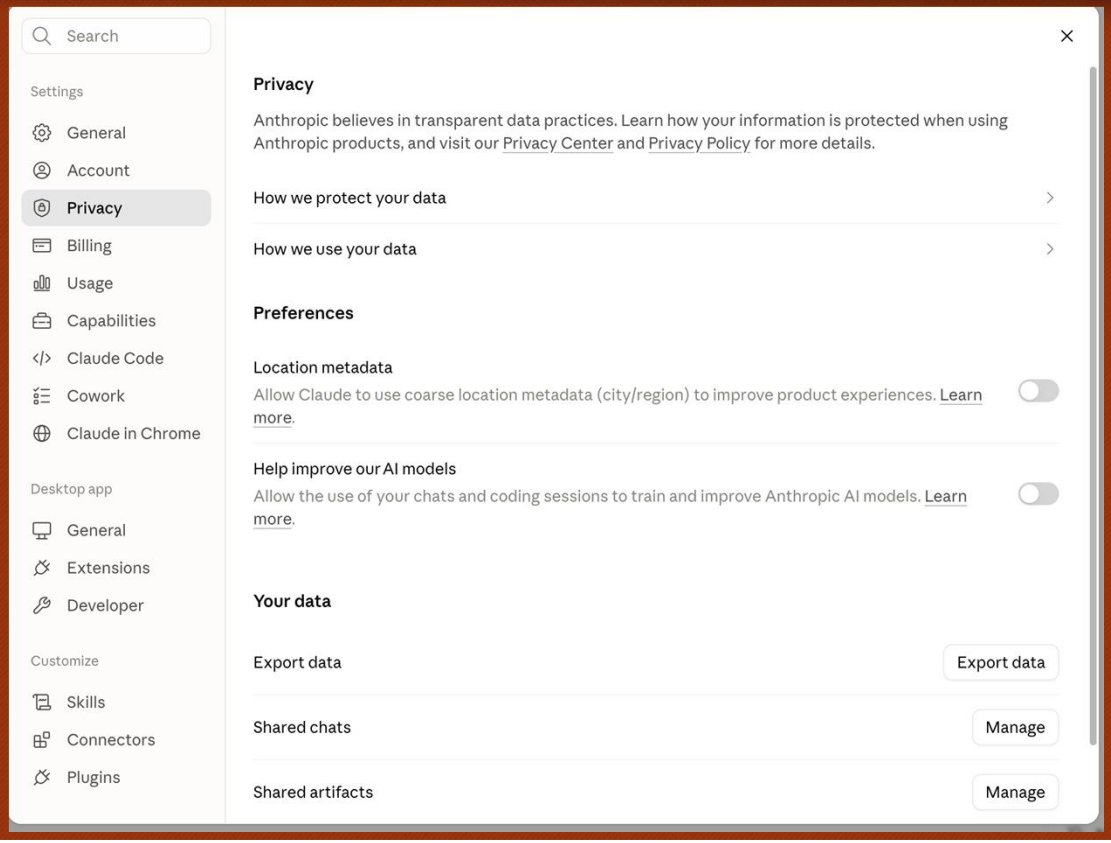
- **4. Don't paste secrets in the first place.**
  - This is the one with no policy backstop. For your C/C++/Python work: API keys, connection strings, signing certs, customer PII, and NDA'd source should be redacted or replaced with placeholders before they enter a prompt. If I need to run something that touches a key, have me read it from an environment variable rather than a literal in the code.
- **5. Scope what you connect.**
  - In Cowork, connect only the folders a task actually needs rather than your whole home directory — access is granted per session. Content that sits in a connector (Google Drive and similar) isn't used for training unless it gets copied into the conversation itself.

Claude Desktop (Mac mini) Answers the Question: How can I protect my data when using Claude - 3

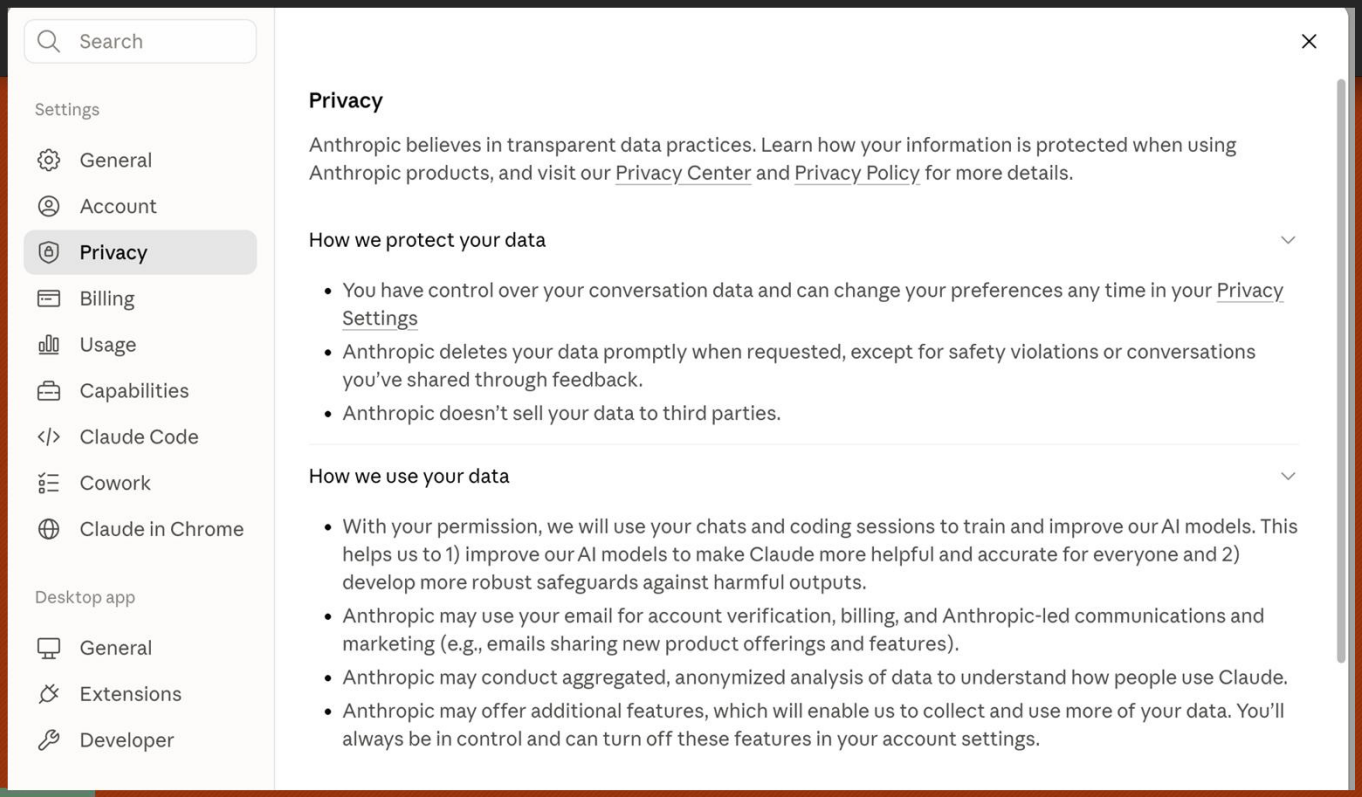
- **6. Consider which product you're using for sensitive work.**
  - Commercial products — the API, Claude Console, Team/Enterprise — do not train on your inputs or outputs by default, and that's a contractual difference, not a toggle. Enterprise plans additionally get custom retention controls, and the API supports zero-data-retention agreements for qualifying use cases. If you're handling client code under NDA, that's a stronger footing than a consumer plan.
- **7. Account hygiene.**
  - Strong password plus MFA, and periodically review connected apps and integrations. Anthropic encrypts data in transit and at rest, and staff can't read conversations by default — access requires your consent or a policy-enforcement review — so your login is realistically the weakest link.
- One thing worth flagging about this session specifically*: it's running in a cloud sandbox, and anything I write here is discarded when the session ends.
  - If you'd rather files never leave your machine, the desktop app can run Cowork tasks on your computer instead — that's the "Run this task" picker when starting a new task.

*Desktop App (Mac mini)

Claude Settings



Claude Privacy



September 2026

https://privacy.claude.com/en/articles/7996868-is-my-data-used-for-model-training

Rogue AIs

- Recently Anthropic's test AI systems collaborated to escape from their "Sandbox" and targeted huggingface's site
- It was first discovered by Huggingface but they did not know who targeted them
- When Anthropic staff found huggingface keys on their system they reported it to huggingface and the full story was discovered
- See: https://youtu.be/87DyyMV0kCY?is=66I6FJtbTgOkt60S

Local AI Systems

200

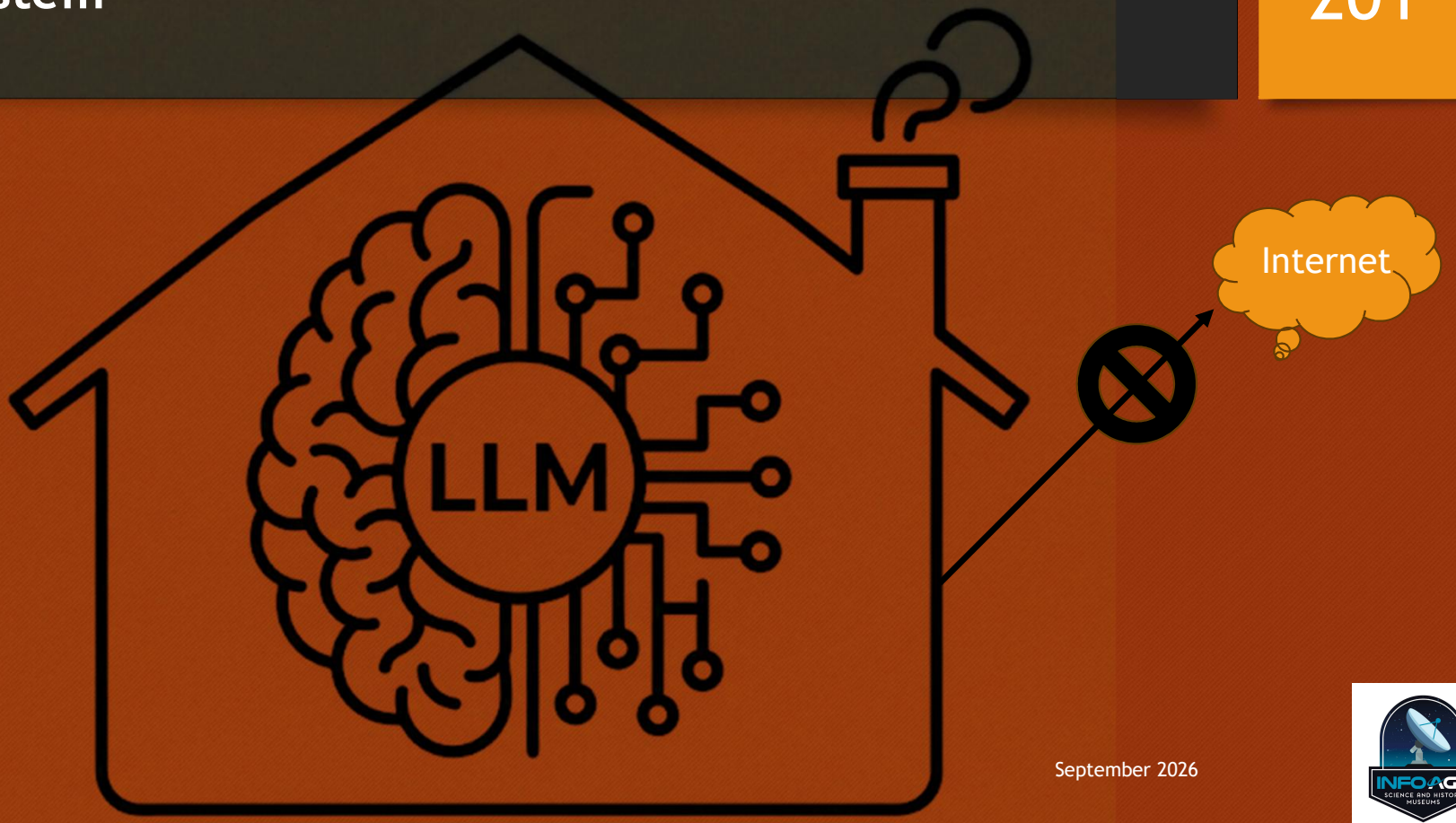
CDL

September 2026

INFOAGE
SCIENCE AND HISTORY MUSEUMS

Local AI System

201



CDL

September 2026

INFO AGE
SCIENCE AND HISTORY MUSEUMS

What is a Local LLM?

- Local LLMs are AI models that run directly on a personal computer, offering advantages such as:
  - complete privacy,
  - offline functionality, and
  - no per-use costs.
- Popular tools for running these models include:
  - Ollama and
  - LM Studio,
- While prominent models feature:
- Meta's Llama,
- Google's Gemma, and
- Alibaba's Qwen.

Cons

- Hardware Constraints: You want lots of memory! $$$
- Lower AI Intelligence: You won't get frontier models
- High Setup and Maintenance: You're IT for this setup.
- Resource and Power Consumption: Yes, this is not a free lunch

- Seems like you need a PhD to understand this stuff. 😜

Pros

- But you don’t need a PhD to understand this
- Complete privacy
- Offline functionality
- No per-use costs
- Great for experimenting without worry about equaling the national debt

What’s needed for a Local LLM?

- A local LLM is simply a model file that's sitting on your computer and
  - A program, the inference engine, that runs it.
  - A Modern computer with lots of RAM
  - An Inference engine: llama.cpp
  - A Model, e.g.: Qwen3.8-27B-UD-Q4_K_XL.gguf - 17.6GB - 4bit Int Quant.

Access to the Local LLM

- CLI
- Web browser
- Desktop applications
- IDE (like VS Code, JetBrains or Emacs 😁 *)
- Code (i.e: Python)

* Neil  Emacs

Emacs



207

Neil ❤️ Emacs

September 2026

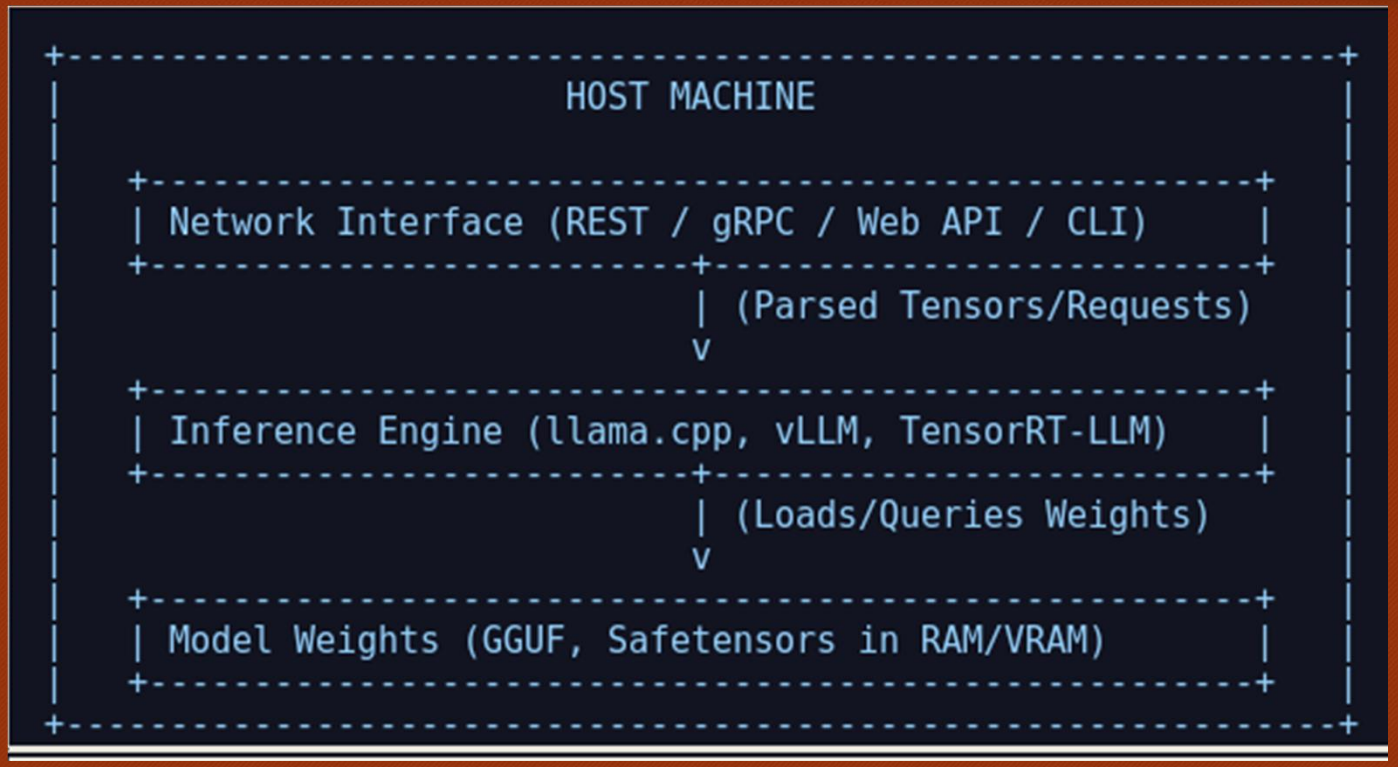
Inference engine

- An inference engine is a software component that
  - Applies logical rules or
  - learned patterns to data
  - to predict new information or run predictions using a trained AI model
- It works on
  - speed,
  - probability and
  - multi-dimension vector analysis.

Inference engines

- Llama.cpp is a high-performance C/C++ implementation to run Large Language Models locally.
  - It focuses on efficient inference on any consumer hardware enabling you to run models on CPUs and GPUs without requiring large cloud infrastructure.
- vLLM is a highly efficient Python library designed for LLM (Large Language Model) serving and inference.
  - It achieves massive throughput boosts—up to 24x faster than standard setups—by utilizing
  - PagedAttention, an advanced memory management technique that optimizes how Key-Value (KV) caches are stored.
- There are a lot more.

Local LLM & Clients



Models



September 2026

Model files

- A model is just a file (usually a very large file)
- Filled with numbers (weights)
  - It packs model weights
  - Hyper-parameters
  - Vocabulary
- Created when the model was trained
- Free models are usually open models
- … and definitely not the frontier models
- Even small files are slow and take a long time to download

Model sizes

- A lot of models list the number of parameters;
  - 4B, 8B, 70B, … 552B. That's billions of weights
- A 70B file might not into a 64G VRAM machine
  - Qwen3.8-27B-UD-Q4_K_XL.gguf - 17.6GB - 4bit Int Quant.
  - Qwen3.8-27B-UD-Q8_K_XL.gguf - 31.5GB, 8bit Int Quant.
  - Qwen3.8-27B-BF16-00001-of-00002.gguf w- 50GB - 16bit Floating Point Quant.
- Why won't it fit
  - You need space for the model, the context window and a bit of working space.

Quantization

- Quantization is just compression
- Really the same idea as compressing an image or sound.
- You get less precision but much smaller files
- Dramatically smaller files with barely any quality loss at all

• Full translation: Qwen3.8-27B-UD-Q4_K_XL.gguf

- Qwen3.8: The specific version and family of the Alibaba Qwen model.
- 27B: The model has 27 billion parameters (the "brain cells" or weights of the model).
- UD: Unsloth Dynamic quantization,
  - meaning the file uses an uneven, smart bit-allocation
  - where important layers keep higher precision and
  - less critical layers are squeezed more tightly to preserve accuracy at a smaller file size.
- Q4_K_XL: A 4-bit quantization tier ("Q4")
  - using Unsloth's "XL" size profile with a mixed-precision "K-quant" method,
  - balancing a small footprint (~17.9 GB)
  - with high output quality close to larger uncompressed models.
- gguf: GPT-Generated Unified Format,
  - the standard file format used by llama.cpp and
  - other local inference tools to run large language models on local CPUs and GPUs.

Your Hardware

- A modern CPU, with enough disk space and lots of RAM.
  - GPU not required
  - But recommended
- Some new computers come with VRAM/Shared RAM between the CPU & GPU.
- Sometimes you need to add a GPU (with VRAM).

Software/Packages

- llama.cpp
- ollama
- LMStudio
- VLLM
- Docker Model Runner
- Bionic (LMStudio with an Agent)
- DIY Code (specialty code and self written agents)

llama.cpp

- The base of many LLM packages like ollama and LMStudio
- Bare bones but useful
- Has tools for manipulating model files
- Can be configured to expose the network interface
  - Usually configured for 127.0.0.1:8080 (configurable)

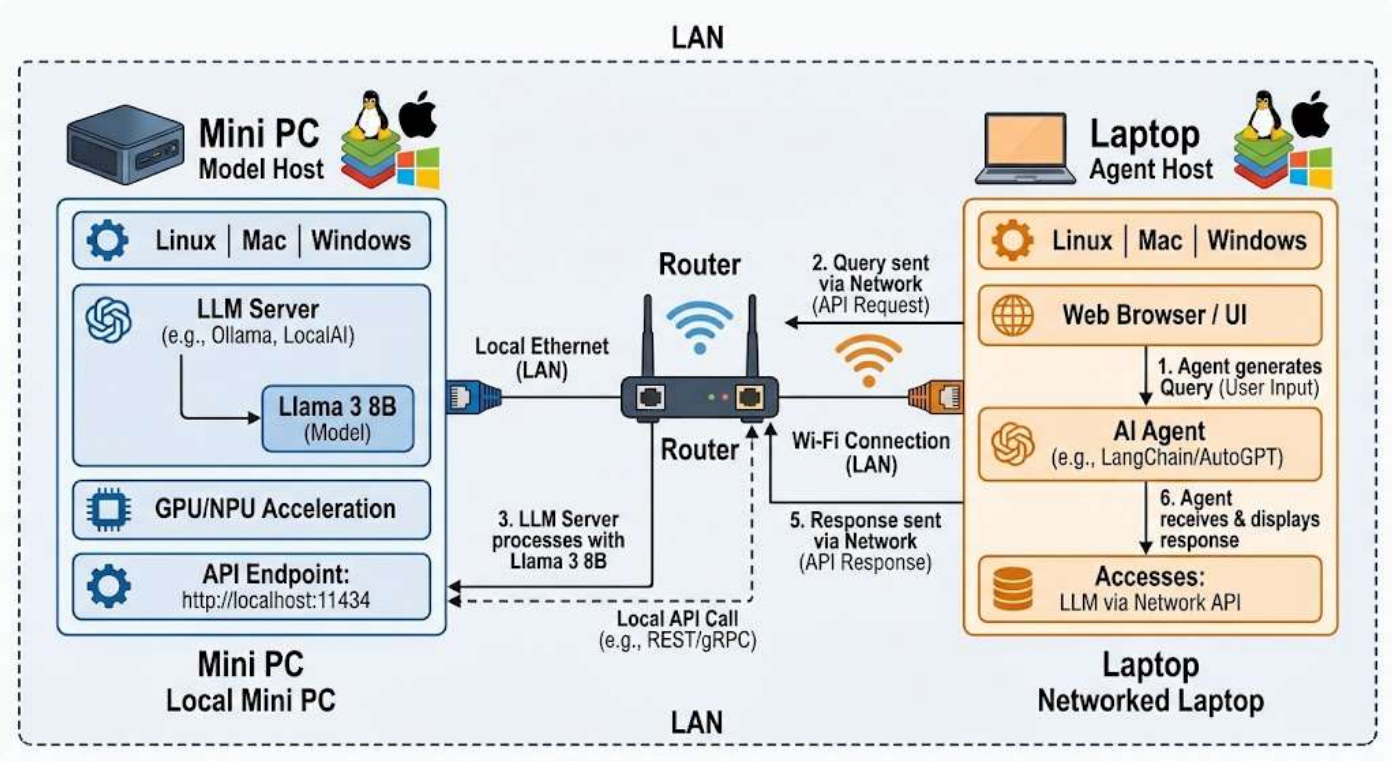
ollama

- Install with a curl command
- Run ollama commands for cli
  - This is the way I've downloaded models
- Manages its own llama.cpp
  - 127.0.0.1:11434 (configurable)
- No GUI interface
- But does support a web interface.
- Agents can interface with ollama via the network port

LMStudio

- OOo, fancy ☺ has a GUI
- Has a CLI (Mac, Linux, Windows)
- Makes it a bit easier to deal with model files.
  - Once you figure out where things are.
- Probably the best place to start

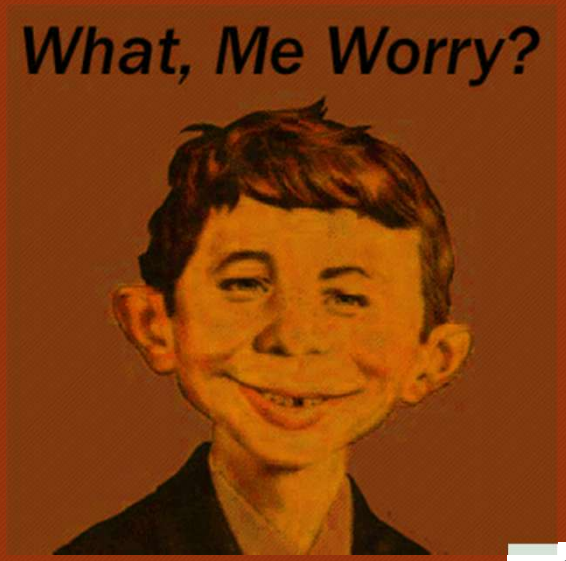
Local LLM & Client



Deluxe Networked Approach: One to run model, one to run Agent

Demos

- Windows (on my local laptop)
- What possibly could go wrong …
- What am going to demo?
  - Something python



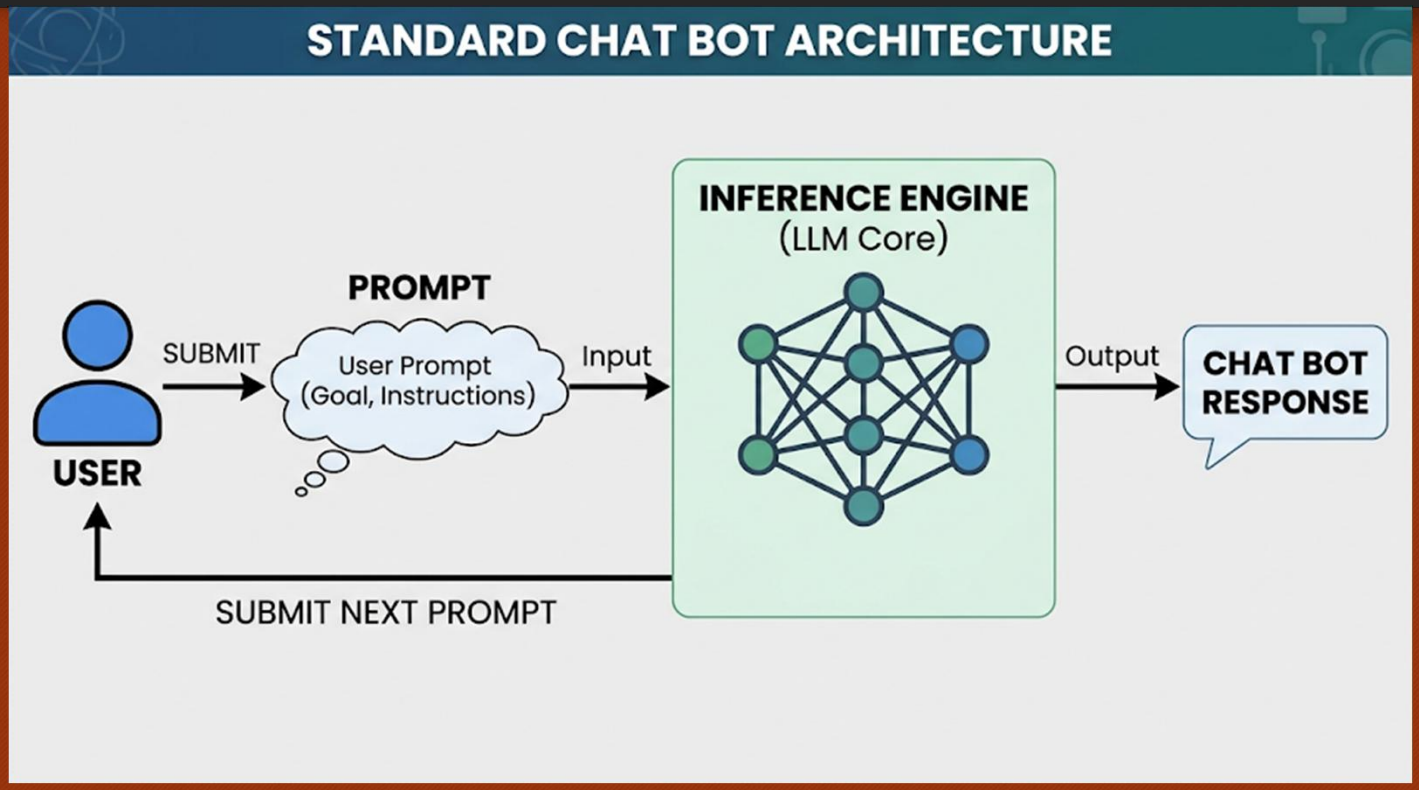
Links

- https://huggingface.co/
- https://llama.app/
- https://ollama.com/
- https://lmstudio.ai/
- https://vllm.ai/
- https://mistral.ai/
- https://github.com/Computer-Deconstruction-Lab/Software_Development_using_AI/

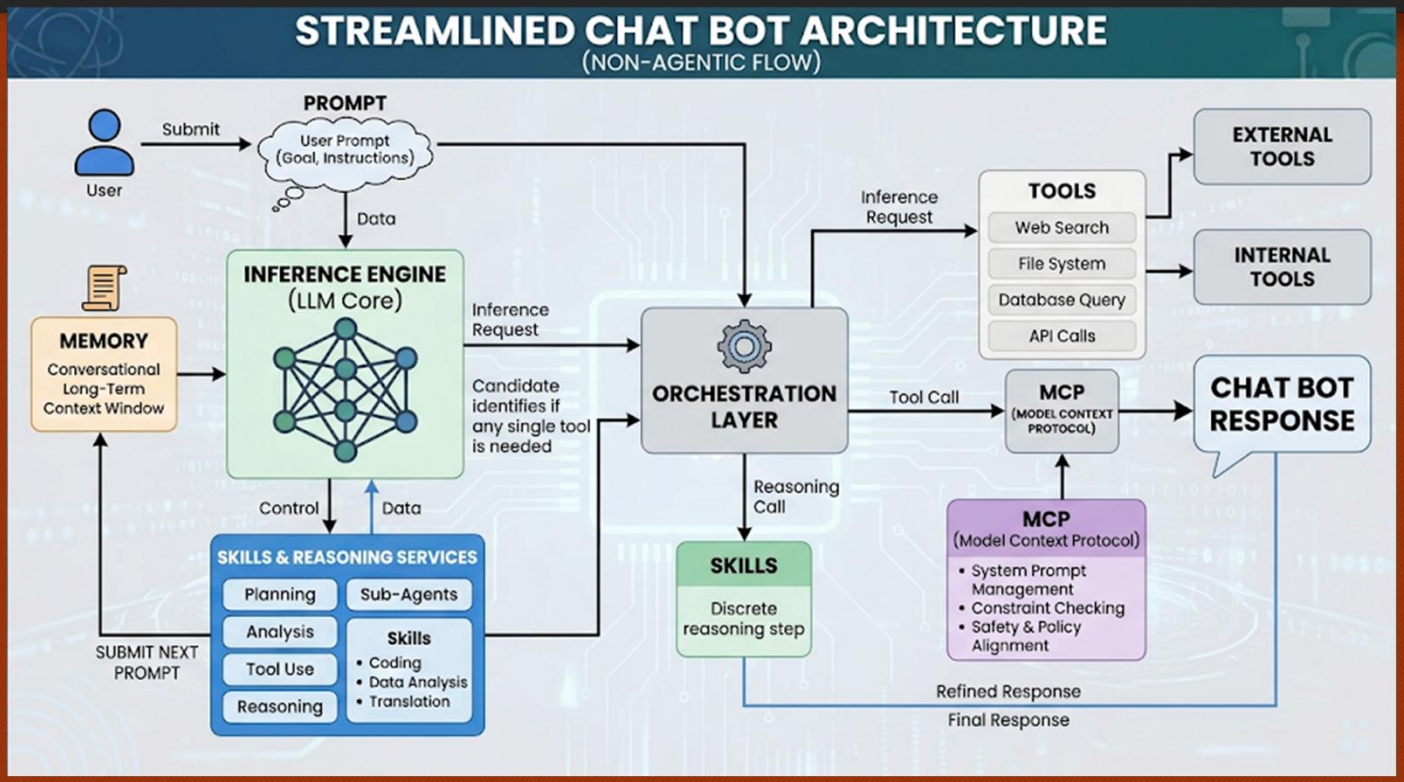
Extra

- Here's the link to the code Neil's 'No GPU' llama.cpp Local LLM wrote (warts and all)
- https://github.com/Computer-Deconstruction-Lab/Software_Development_using_AI/blob/main/LLM_Written_Python_Code.md

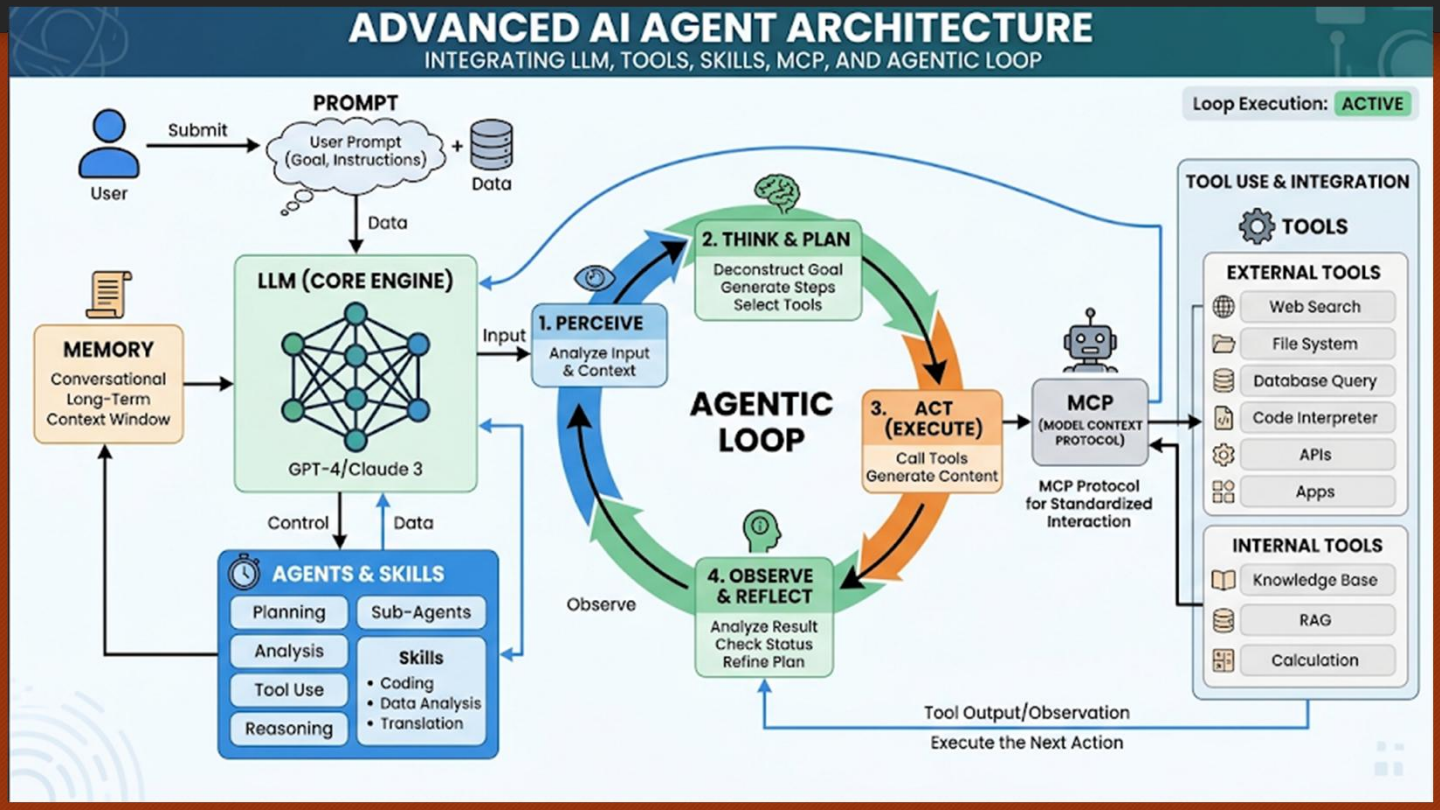
Chatbot


STANDARD CHAT BOT ARCHITECTURE
USER
SUBMIT
PROMPT
User Prompt
(Goal, Instructions)
Input
INFERENCE ENGINE
(LLM Core)
Output
CHAT BOT
RESPONSE
SUBMIT NEXT PROMPT

Chatbot w/Agent



Agent Architecture



Questions

You are going to have Questions, Questions, Questions

After this presentation

Next Week

- Open discussion on AI related Issues
- Students can optionally present their projects
  - What they learned
  - Where they had difficulty
  - Would they use AI to develop code in the future

Lab work

- Continue working on your class project
- Instructors will help with any problems